

AI4SWENG — Observability Integration Guide

This guide explains how to connect KIO microservices to the central AI4SWENG observability platform. By integrating OpenTelemetry and Langfuse into these services, you allow the platform to monitor orchestrator and worker health, request latency, execution paths across services, and LLM token usage.

1. High-Level End-to-End Flow

This section represents the high-level execution cycle with telemetry routing across the KIO microservices.

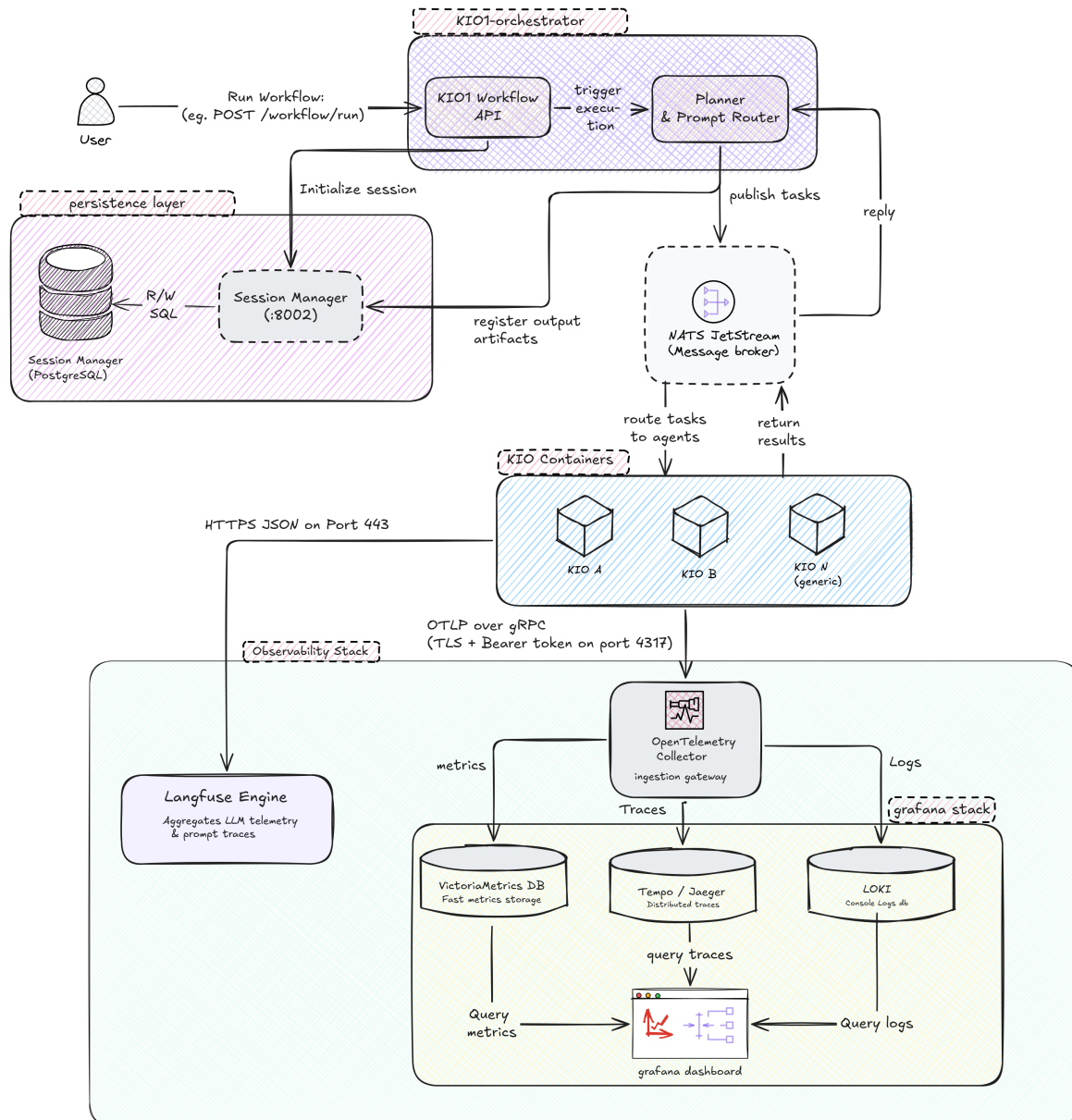


Figure 1: High-Level End-to-End Flow Diagram

A. End-to-End Execution Process (Step-by-Step)

- **Workflow Triggering:** A client initiates a task by sending an HTTP request (e.g. `POST /workflow/run`) to the `KIO1 Workflow API`.
- **Session Initialization:** The `Workflow API` immediately calls the `Session Manager` microservice to register the start of this execution. The `Session Manager` writes a new session record into the `PostgreSQL Database` and generates a unique `session_id`. The API returns this ID to the client with an HTTP 202 `Accepted` status so the client can track progress.
- **Task Routing:** The `Planner & Prompt Router` takes over the execution. It constructs the workflow graph and publishes the first task payload over NATS JetStream. This payload is wrapped inside a standardized `KIOEnvelope` schema that carries the active `session_id` and the target `kio_id`.
- **Agent Execution:** NATS JetStream routes the envelope to the appropriate worker container (e.g., `KIO5`). The worker container's application code extracts the `session_id` from the envelope and begins executing the task (e.g. running LLM completions or code execution tools).
- **Telemetry Transmission:** During execution, the worker container actively transmits telemetry over two network streams:
 - **OpenTelemetry Stream:** The OpenTelemetry SDK aggregates metrics and traces in-memory and pushes OTLP data over a secure gRPC connection (port 4317) to the central `OTel Collector`.
 - **Langfuse Stream:** The Langfuse SDK captures prompt inputs, completions, token usage, and costs, pushing them directly to the `Langfuse Server` over HTTPS (port 443).

In addition to these active network streams, the container prints logs to `stdout/stderr`. The `OTel Collector` passively scrapes these console logs from the container host and routes them to `Loki`.

- **Telemetry Routing:** The `OTel Collector` splits the incoming OpenTelemetry metrics/traces and the scraped console logs, writing them to their respective databases: metrics to `VictoriaMetrics`, traces to `Tempo`, and logs to `Loki`.
- **Task Completion:** Once the worker container finishes its task, it publishes the results and output artifacts back to NATS JetStream.
- **Lineage Registration:** NATS routes the worker's reply back to the `LangGraph Planner`. The planner reads the results and calls the `Session Manager` to write the output artifacts and register their lineage history into the `PostgreSQL Database`.

B. The Observability & Grafana Stack Components

- **VictoriaMetrics:** A highly optimized time-series database. It stores all numeric metrics (like CPU usage, request counts, and execution durations) and is queried by Grafana using the PromQL query language.
- **Tempo:** A distributed tracing storage backend. It records timing spans showing the exact execution timeline and code path across multiple containers, and is queried by Grafana using the TraceQL query language.
- **Loki:** A log aggregation database designed specifically for container console prints. It indexes search metadata labels (like `session_id` and `kio_id`) rather than the full log text, keeping text storage fast and cost-effective, and is queried by Grafana using the LogQL query language.
- **Grafana Dashboard:** The single web portal that connects all three databases. It queries `VictoriaMetrics`, `Tempo`, and `Loki` dynamically based on the dropdown filter variables, drawing the graphs and log viewer on a single interactive screen.

2. In-Process SDK Flow

This section describes the internal telemetry lifecycle inside the memory space of a single KIO worker container (e.g., KIO5) as it processes a task.

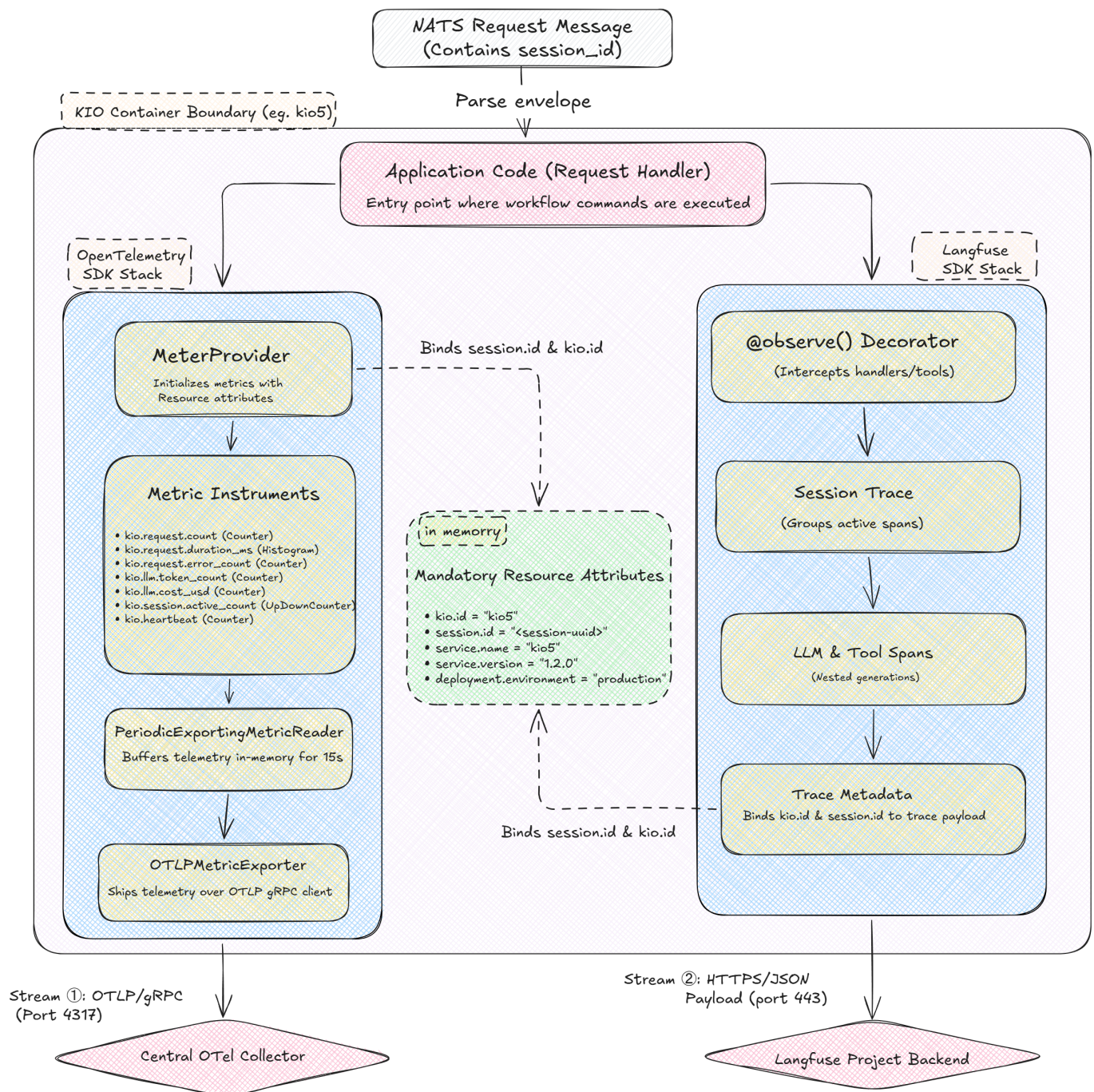


Figure 2: In-Process SDK Flow Diagram

A. The In-Process Execution Process (Step-by-Step)

The flow within the container executes as follows:

1. **Request Ingestion:** The container's application entry point (**Application Code**) receives a task message from NATS JetStream. This message is wrapped inside the **KIOEnvelope** containing the **session_id**.
2. **Context Extraction:** The application handler extracts the **session_id** string from the incoming envelope.
3. **OpenTelemetry Initialization:** The container's OTEL **MeterProvider** reads the **session_id** from the request, along with the container's static environment properties (**kio.id**, **service.name**, **service.version**, and **deployment.environment**), and binds them to the active **Mandatory Resource Attributes** namespace in memory.
4. **Metric Recording:** As the handler code runs, it updates the OTEL **Metric Instruments** (e.g. incrementing the request counter, recording durations, or setting active session concurrency).
5. **Metric Buffering & Export (Stream 1):** The **PeriodicExportingMetricReader** collects these recorded metrics and buffers them in memory for up to 15 seconds. When the window expires, the **OTLPMetricExporter** packages the buffer and pushes it over a secure gRPC connection (port 4317) to the OTEL Collector.
6. **Langfuse Interception:** Simultaneously, the **@observe()** decorator intercepts the function entry point. It creates a new **Session Trace** bucket in-memory and associates it with the active **session_id**.
7. **Recording AI Operations:** As the container executes internal prompt logic, child decorators capture the prompt text, model details, token consumption, and execution durations for each LLM and tool call as child **LLM & Tool Spans**.
8. **Stamping Metadata:** During execution, the code calls **langfuse_context.update_current_trace** to stamp the **session_id** and static **kio_id** as **Trace Metadata** on the trace payload, matching the exact attributes bound in OpenTelemetry.
9. **Langfuse Payload Export (Stream 2):** When the handler completes, the Langfuse SDK serializes all spans and metadata into a JSON payload and transmits it directly to the Langfuse Server via HTTPS on port 443.

3. Grafana Dashboard Layout

This page showcases the central Grafana dashboard designed to visualize the telemetry collected from KIO microservices. The dashboard layout is organized into rows covering system liveness and concurrency, request performance and volumes, error tracking and raw console logs, and LLM operational economics, dynamically filtered by container ID and session UUID variables.



Figure 3: Grafana Dashboard Layout

4. Onboarding Steps

Step 1: Request Credentials - contact the platform team for your **OTLP Bearer Token** and **Langfuse Project Keys** (Public/Secret).

Step 2: Configure Environment Variables - set these variables in your container environment:

```
1 OTEL_EXPORTER_OTLP_ENDPOINT="https://otel-collector.ai4sweng.io:4317"
2 OTEL_EXPORTER_OTLP_HEADERS="Authorization=Bearer <YOUR_OTLP_TOKEN>"
3 OTEL_RESOURCE_ATTRIBUTES="service.name=kio5,service.version=1.2.0,kio.id=kio5,deployment.environment=production"
4 LANGFUSE_PUBLIC_KEY="pk-lf-..."
5 LANGFUSE_SECRET_KEY="sk-lf-..."
6 LANGFUSE_HOST="https://langfuse.ai4sweng.io"
```

Step 3: Install Required Packages - add: opentelemetry-api, opentelemetry-sdk, opentelemetry-exporter-otlp-proto-grpc, and langfuse to your project.

Step 4: Initialize SDKs in Code - initialize the global MeterProvider at startup.

Step 5: Verify Connectivity - run a test and verify that `kio.heartbeat` shows on Grafana and traces show in Langfuse.

5. Telemetry details

Mandatory Metrics Schema (every metric must carry the `kio.id` label):

Metric Name	Type	Unit	Key Description & Metric Labels
<code>kio.request.count</code>	Counter	1	Total requests processed (labels: <code>status="ok" "error"</code>)
<code>kio.request.duration.ms</code>	Histogram	ms	End-to-end request execution latency
<code>kio.request.error.count</code>	Counter	1	Total error counts (labels: <code>error.type</code>)
<code>kio.llm.token.count</code>	Counter	tokens	Input/Output tokens processed (labels: <code>direction</code>)
<code>kio.llm.cost.usd</code>	Counter	USD	Estimated LLM API transactional cost
<code>kio.session.active.count</code>	UpDownCounter	1	Concurrent active requests inside container
<code>kio.heartbeat</code>	Counter	1	Container liveness tick (every 60 seconds)

Rules and Conventions:

- **Metric naming:** use dot-notation (`kio.<domain>.<metric>`). Do not change metric names once they are established.
- **Low cardinality for metric labels:** do not use unique values like session IDs, usernames, or full URLs as metric labels. Doing so degrades the performance of the VictoriaMetrics database. Use session IDs only in trace metadata.
- **Secrets and PII:** never include API keys, passwords, email addresses, or personal data in metric labels or Langfuse payloads.
- **Heartbeat task:** run the `kio.heartbeat` counter in a background thread or loop. If the platform does not receive a heartbeat for more than 120 seconds, your service will be flagged as offline.

6. Appendix — Reference Implementation

This Python snippet registers a globally configured OpenTelemetry `MeterProvider` with periodic OTLP gRPC export, and wraps the request logic with Langfuse tracing.

```

1 import os, time, asyncio
2 from opentelemetry import metrics
3 from opentelemetry.sdk.metrics import MeterProvider
4 from opentelemetry.sdk.metrics.export import PeriodicExportingMetricReader
5 from opentelemetry.exporter.otlp.proto.grpc.metric_exporter import OTLPMetricExporter
6 from opentelemetry.sdk.resources import Resource
7 from langfuse.decorators import observe, langfuse_context
8
9 # 1. OpenTelemetry Setup
10 resource = Resource.create({
11     k: v for attr in os.environ.get("OTEL_RESOURCE_ATTRIBUTES", "").split(",")
12     for k, v in [attr.split("=")] if "=" in attr
13 })
14 exporter = OTLPMetricExporter(insecure=False)
15 reader = PeriodicExportingMetricReader(exporter, export_interval_millis=15000)
16 metrics.set_meter_provider(MeterProvider(resource=resource, metric_readers=[reader]))
17 meter = metrics.get_meter("kio.instrumentation")
18
19 # 2. Mandatory Instruments
20 request_counter = meter.create_counter("kio.request.count", unit="1")
21 request_duration = meter.create_histogram("kio.request.duration_ms", unit="ms")
22 error_counter = meter.create_counter("kio.request.error_count", unit="1")
23 llm_token_counter = meter.create_counter("kio.llm.token_count", unit="tokens")
24 llm_cost_counter = meter.create_counter("kio.llm.cost_usd", unit="USD")
25 active_sessions = meter.create_up_down_counter("kio.session.active_count", unit="1")
26 heartbeat_counter = meter.create_counter("kio.heartbeat", unit="1")
27 KIO_ID = os.environ.get("KIO_ID", "kioN")
28
29 # Background heartbeat task
30 async def _heartbeat_loop():
31     while True:
32         heartbeat_counter.add(1, {"kio.id": KIO_ID})
33         await asyncio.sleep(60)
34
35 # Request handler wrapper
36 async def handler(envelope):
37     session_id = envelope.get("session_id", "unknown")
38     labels = {"kio.id": KIO_ID}
39     active_sessions.add(1, labels)
40     start = time.monotonic()
41     try:
42         result = await _do_llm_work(session_id, envelope)
43         request_counter.add(1, {**labels, "status": "ok"})
44         request_duration.record((time.monotonic() - start) * 1000, labels)
45         return result
46     except Exception as exc:
47         request_counter.add(1, {**labels, "status": "error"})
48         request_duration.record((time.monotonic() - start) * 1000, labels)
49         error_counter.add(1, {**labels, "error_type": "internal"})
50         raise
51     finally:
52         active_sessions.add(-1, labels)
53
54 @observe(name="llm_call")
55 async def _do_llm_work(session_id: str, envelope: dict):
56     langfuse_context.update_current_trace(
57         session_id=session_id,
58         metadata={"kio.id": KIO_ID, "session.id": session_id},
59     )
60     # LLM completion call goes here...

```